

machine implementations.

Julia provides a wide variety of options for building ML solutions by offering various libraries. Not just for ML, Julia in general is very rich in terms of the libraries and packages it offers. A complete list is provided in the Julia Observer page (<https://juliaobserver.com/>). Even the list of libraries that Julia offers to build ML solutions alone is very long, as shown in Figure 1.

This article provides a brief introduction to the following libraries in Julia:

- Flux
- Knet
- MLBase.jl
- TensorFlow.jl
- ScikitLearn.jl

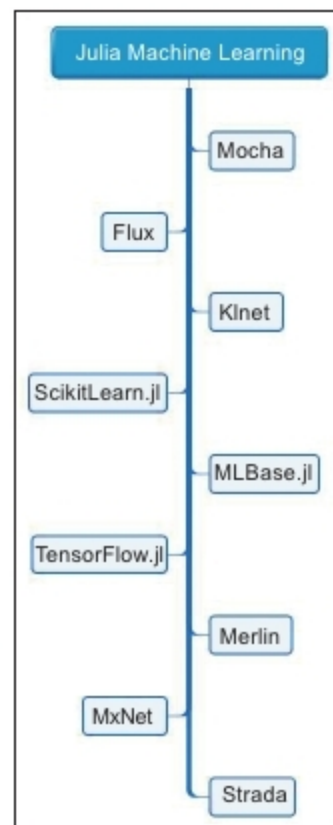


Figure 1: Julia machine learning options

Flux

Flux is a popular choice for building machine learning solutions in Julia. This can be inferred from the Julia Observer Web page, which has higher ratings for Flux. The following are the advantages of using Flux:

- It is a very lightweight option.
- The entire Flux code is written in Julia. This is unlike libraries in some other languages for which the interface is in one language and the core components are in another such as C++.
- Another big advantage of Flux is that it comes with many built-in tools. It is fully loaded with features that are required for building a ML solution.
- Flux can be used along with other Julia components such as data frames and differential equation solvers.

You can start using Flux straight away without complex installation processes. You just need to use *Pkg.add*, as shown below:

```
Pkg.add("Flux")
```

As the official documentation says, Julia is simply executable math, just like Python is the executable pseudo code. Hence the conversion of mathematical expressions into working code is elegant in Julia.

To start with, there are some executable Flux experiments listed in the <https://fluxml.ai/experiments/> page. You can navigate and see the live demos to get an insight into the power of Flux. For example, a style transfer demo application with the cover page of *OSFY* magazine is shown in Figure 2.

The official documentation (<https://fluxml.ai/Flux.jl/stable/>) lists the complete set of features provided by Flux.

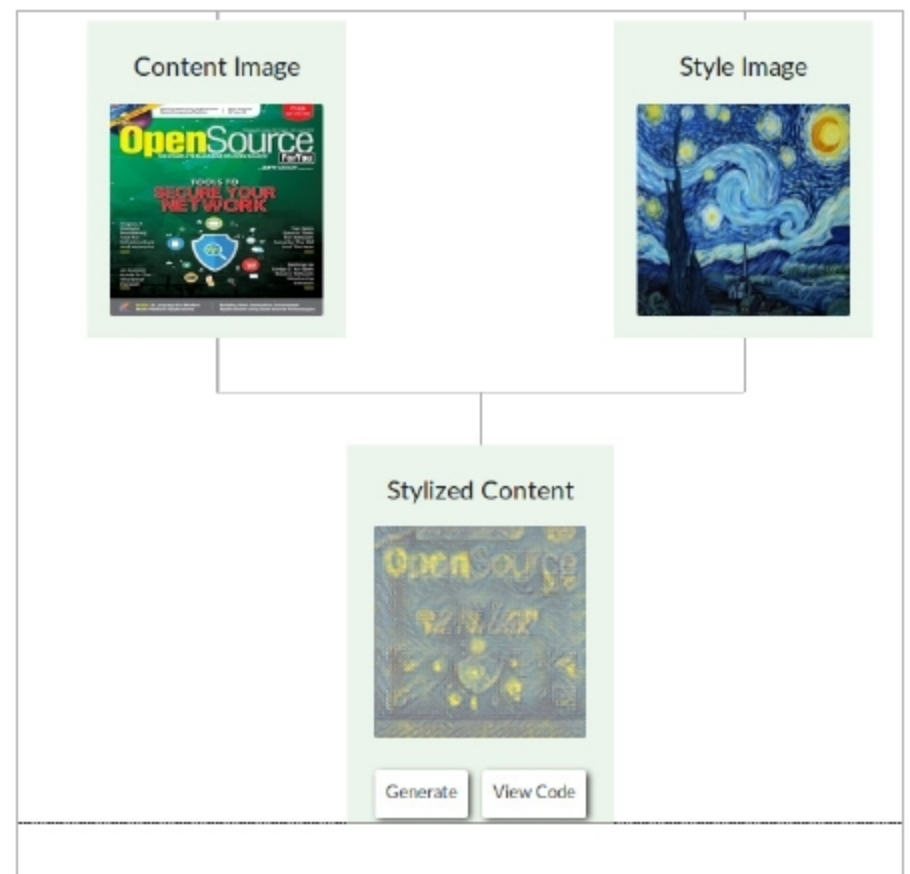


Figure 2: Style transfer demo with *OSFY* cover page

The Model-Zoo has various demonstrations which can be very useful in understanding the many dimensions of Flux (<https://github.com/FluxML/model-zoo/>). These models are categorised into the following folders:

- Vision: Large Convolutional Neural Networks (CNNs)
- Text: Recurrent Neural Networks (RNNs)
- Games: Reinforcement learning (RL)

If you are working in the field of computer vision, you may find Metalhead an interesting option. This package provides computer vision models that run on top of the Flux library. A sample code sequence to perform image content recognition is shown in Figure 3.

Overall, Flux is evolving into a major choice for ML development in Julia.

Knet

Knet has been built by Koc University, Turkey. It is a



Figure 3: Flux computer vision code sequence